MCP 协议级安全审计

日期：2026-10-24

审计器：CBAP Kernel / DeviceDriver 的 idempotency、lease、facts、stop 四个原语

对象：Model Context Protocol 规范与公开 issue 轨迹

规范版本：`2026-07-28`（仓库 commit `dae694f`）

历史对照：`2025-11-25`（公开 issue #3394 的复现环境）

性质：协议层安全/安全相关一致性审计；不是渗透测试，不是 CVE 认定

---

0. 执行摘要

用 CBAP 的验收原语检查 MCP 后，**确认了一个协议级的安全相关缺口**：

> 一个 mutating `tools/call` 已经提交外部副作用，但响应丢失；客户端用新的 JSON-RPC `id` 重试时，MCP 没有标准的幂等/去重语义，副作用可以执行两次。

这不是“某个 server 造假”，也不是 JSON-RPC 实现 bug。

它是**协议没有定义 lost-response 后的效果语义**。

已在公开 issue 中有人提出：

```
modelcontextprotocol/modelcontextprotocol#3394
Lost-response retry on a mutating `tools/call` re-executes external effects
```

本报告做的是：

```
用 CBAP 的最小模型独立复现
检查当前规范是否已经补上语义
判断它属于哪一层问题
给出 CBAP 侧可用的缓解
```

没有发现可确认为 RCE、认证绕过或越权的协议级漏洞。

不建议把这个问题直接发成公开“安全漏洞”；它已经在公开 issue 中，且更准确的分类是：

```
安全相关的完整性/可靠性缺口
高影响场景：物理设备、支付、写入、开门、给药、移动
```

---

1. 范围与方法

1.1 CBAP 的四个检查原语

```
idempotency   同一个逻辑操作重复提交，是否只产生一次效果
lease         共享设备是否有单一所有者
facts         动作后是否有独立于 tool 回执的后置事实
stop          是否能要求终止在途动作，并得到安全状态
```

MCP 当前对应的机制：

```
idempotency   idempotentHint（非规范性提示）
lease         无协议级设备租约
facts         tool result / structuredContent，但不是设备事实
stop          notifications/cancelled，best-effort
```

1.2 审计边界

```
规范文本
JSON-RPC 消息
stdio 传输
一个最小 mutating MCP server
CBAP 现有幂等测试
公开 issue #3394 的复现记录
```

没有测试：

```
OAuth 授权服务器实现
真实公网 MCP 服务
浏览器/客户端 UI
第三方 SDK 的全部传输
```

2. 发现

ID	发现	级别	性质
P-01	lost-response 后重试可重复执行副作用	高（物理/写入场景）	协议语义缺口
P-02	`idempotentHint` 是非规范性提示，不能作为重试依据	中	语义误导风险
P-03	cancellation 是 best-effort，没有终止确认/安全状态	中高	安全能力缺口
P-04	tool success 没有标准的后置事实契约	中	可验证性缺口
P-05	没有协议级设备 lease/owner	中	并发安全缺口
P-06	tool 定义可在用户批准后变更，无强制 pin/重批准	高（Agent 场景）	已知 SEP #3452
P-07	`server/discover.instructions` 是可进入 LLM 上下文的字符串，未标记 untrusted	中高	字符串来源缺口
P-08	跨 server 的 tool 名冲突只要求 SHOULD 消歧	中	工具影子攻击面

P-01：lost-response 重复副作用

最小序列：

```
1. client -> tools/call(bump), id=2
2. server 提交 effect: count=1
3. server 的响应丢失
4. client 超时
5. client 用新的 JSON-RPC id=3 重试
6. server 再次执行 bump: count=2
```

实测输出：

```
{"first_error": "MCP timeout: tools/call",
 "state_after_first": {"count": 1},
 "retry_fresh_id": {"ok": true, "count": 2},
 "state_after_retry": {"count": 2}}
```

关键点：

```
effect 在响应丢失前已经提交
重试是新的 JSON-RPC id
server 没有稳定逻辑操作键可去重
规范没有要求 server 识别这是同一次逻辑操作
```

P-02：`idempotentHint` 不能救命

MCP schema 里有：

```
readOnlyHint
destructiveHint
idempotentHint
```

它们都是**注解/提示**，不是双方必须执行的幂等协议。

客户端不能因为 `idempotentHint=true` 就自动重试一个 mutating call；

server 也不能因为该字段为 true 就跳过重复检测。

P-03：cancellation 不是 stop

当前取消语义：

```
client SHOULD send notifications/cancelled
server SHOULD stop processing
server MAY ignore if already completed / cannot cancel
```

没有：

```
终止确认
安全状态回执
设备已停/未停的事实
```

对物理设备，`cancelled` 不等于 `stopped`。

P-04：tool result 不是 postcondition

MCP 的 tool result 可以包含 `structuredContent`，但没有标准要求它必须是**独立设备事实**。

`{"ok": true}` 完全合法，但它不能证明：

```
门锁了
药注射了
机器人停了
款付了
```

P-05：没有设备租约

MCP 没有协议级的：

```
lease
owner
fencing token
```

多个客户端可以同时调用同一个有状态设备。

这不是当前 MCP 的攻击面定义，但对物理控制是实际风险。

P-06：工具定义可以在批准后改变

MCP 的 `tools/list` 结果可以因为 `listChanged` 被重新获取。

当前规范没有强制客户端：

```
保存 description 的摘要
比较重新获取后的 description
变化时重新征求用户批准
```

攻击序列：

```
1. server 先给一个正常工具描述
2. 用户批准/缓存
3. server 发 tools/list_changed
```

```
4. 重新获取时换成带指令的描述
5. 客户端把新描述原样送进 LLM 上下文
```

这不是某个 server 的 bug；是**能力目录没有强制完整性**。

已有公开 SEP：

```
#3452  Mandatory tool definition integrity — description pinning and content validation
```

SEP 的 F-01/F-02 正是：

```
F-01 rug pull
F-02 tool description injection
```

CBAP 的做法是：

```
device manifest 固定 tools / transport / facts
capability digest 变更 -> 拒绝，直到重新审批
```

MCP 目前没有这个强制层。

这条不应重复开新 issue；要补就补到 #3452 的 SEP 讨论里。

P-07：`instructions` 也是可注入的字符串

`server/discover` 的结果里有：

```
instructions?: string
```

规范描述它是给 LLM 的自然语言使用说明。

但规范没有要求：

```
标记它为 untrusted
首次接收后 pin 住
变化时重新批准
与 tool 描述做同样的注入扫描
```

这意味着一个 server 可以在发现阶段就把指令送进模型上下文，

而不需要调用任何 tool。

这不是“tool 描述注入”的重复，而是**另一个字符串入口**。

CBAP 侧建议：

```
server instructions = 数据，不是系统指令
默认隔离；只有显式批准后才进入模型
按 server + version 做摘要
```

这条目前没有看到专门的 SEP；可以作为候选 issue，但先不要在公开仓库重复造轮子。

P-08：跨 server 的 tool 影子

规范明确：

```
tool name 只在单个 server 内唯一
serverInfo.name 不保证全局唯一
聚合器 SHOULD 做前缀/消歧
```

但 `SHOULD` 不是 `MUST`。

一个聚合器如果把两个 server 的 `search` 直接暴露给模型，

模型无法知道它调用的是哪个 server 的 `search`。

更危险的是描述注入：

```
server A 的工具描述可以指示模型去调用 server B 的工具
```

MCP 本地安全指南已经把它列为 tool poisoning / tool shadowing；

协议层目前只给 SHOULD 级建议。

CBAP 的对应做法：

```
device_id 命名空间
tool 名永远带 server 前缀
调用前核验 owner lease
```

2.7 字符串通道矩阵

CBAP 把“能发字符串的东西”都当作潜在设备；MCP 的字符串通道至少包括：

通道	来源	是否标记 untrusted	是否有 pin/摘要	CBAP 处理
tool `description`	server	否（仅注解有警告）	否（#3452 提议）	manifest + digest
`server/discover.instructions`	server	否	否	隔离/审批
tool result `content`	server/device	否	否	独立 facts 复核
resource content	server/device	否	否	数据，不执行
error message	server	否	否	只记录，不当作指令
`resource_link.uri`	server	否	否	出站 allowlist
`icons.src`	server	同源校验	否	不自动抓取
prompt content	server/user	取决于来源	否	用户选择

这张表的核心结论：

`MCP 没有统一的“字符串来源/信任级别”字段`

对纯文本聊天无所谓；

对能操作物理世界、文件、网络、支付的 CBAP 代理，这是必须补的层。

---

3. 规范与 issue 轨迹

3.1 当前规范

在 `2026-07-28` 中检索：

`idempotency / idempotent / retry / exactly-once / duplicate`

没有找到针对 `tools/call` lost-response 的规范性重试/去重语义。

相反，MRTR 的 retry 明确要求：

```
The JSON-RPC id MUST be different between the initial request and the retry.
```

这使 server 无法用 JSON-RPC id 识别“同一次逻辑操作”。

3.2 cancellation

规范对取消使用：

```
SHOULD stop processing
MAY ignore
```

并允许已完成的请求不响应。

这与 CBAP 需要的“stop 后必须能读到 safe state”不是同一件事。

3.3 公开 issue

```
https://github.com/modelcontextprotocol/modelcontextprotocol/issues/3394
```

状态：

```
OPEN
作者：arjun2075
创建：2026-09-27
最近更新：2026-10-23
```

该 issue 的结论边界与本报告一致：

```
不是协议违规
是规范没有定义责任边界
应用层幂等可以缓解
不应从 idempotentHint 推断重试安全
```

不要重复开新 issue。

如果要补充，应该在该 issue 下给出新的独立证据或具体规范文本建议。

---

4. CBAP 侧缓解

4.1 同 key 只执行一次

CBAP 现有回归：

```
tests/cbap/test_conformance.py::test_12_idempotent_write
```

同一 `idempotency_key` 写两次：

```
side effect count = 1
```

这说明缓解必须在**效果边界**实现，而不是在 JSON-RPC id 上实现。

4.2 CBAP 的推荐组合

```
逻辑操作 id     由上层生成，重试时保持不变
idempotency     由效果层持久化，重复请求返回原结果
lease           共享设备同一时刻只有一个 owner
facts           动作后读独立状态，不把 ok 当完成
stop            单独能力；stop 后读 safe_state
```

4.3 不能做的事

```
不能靠 JSON-RPC id 去重（规范要求新 id）
不能靠 idempotentHint 去重（只是提示）
不能靠 notifications/cancelled 证明已停止
不能靠 tool result 证明设备状态
```

---

5. 影响判断

5.1 适用场景

高风险：

```
物理执行器
门禁/锁
医疗设备
支付/转账
写入/删除
车辆/机器人运动
```

中风险：

```
发消息
创建工单
提交表单
资源分配
```

低风险：

```
只读查询
可重复的幂等查询
```

5.2 攻击面

不需要 server 恶意。

以下情况都能触发：

```
网络丢包
反向代理超时
客户端超时后自动重试
网关重试
进程崩溃前的已提交效果
```

如果攻击者能丢弃响应但不能伪造请求，仍可能诱导**重复副作用**。

这不是机密性漏洞，是完整性和物理安全问题。

---

6. 建议

6.1 对 MCP 规范

最小、非破坏性的补充：

```
1. 在 tools/call 文档中明确：
   lost response 后结果不确定；自动重试 mutating call 可能重复执行效果。
2. 明确 idempotentHint 不是重试授权。
3. 可选：定义稳定的 idempotency key / effect id 字段。
4. 可选：定义 cancellation 的 terminal state / safe_state。
```

不建议：

```
直接声称 exactly-once
把设备 stop 语义塞进核心协议
在没有实现的情况下强制所有 server 支持
```

6.2 对客户端

```
不要自动重试 mutating tools/call
重试前先查 facts / 读回状态
无法确认时标记 outcome_unknown
使用稳定逻辑操作键
```

6.3 对 server / 设备

```
效果层持久化 idempotency key
重复请求返回原结果
暴露独立状态读取
提供真正的 stop 能力
```

---

7. 披露建议

当前不建议走 GitHub Security Advisory，原因：

```
没有确认协议实现漏洞
没有越权/ RCE / 数据泄露
公开 issue #3394 已存在
本质是规范语义缺口
```

建议：

```
在 #3394 下补充独立复现或规范文本建议
或提交 SEP，明确 idempotency/retry 责任边界
```

如果发现的是**实现层可利用漏洞**（例如某个 SDK 自动重试并导致越权/重复扣款），

那应该：

```
走私密 Security Advisory
不在公开 issue 里放利用细节
```

---

8. 复现材料

8.1 最小 mutating server

```
tests/cbap/fake_mcp_mutator.py
```

行为：

```
bump: count += 1; 第一次响应可丢弃
get_state: 读取 count
```

8.2 最小复现

```
python3 - <<'PY'
from drivers.mcp_client import MCPStdioClient
import sys
c = MCPStdioClient([sys.executable,
    'tests/cbap/fake_mcp_mutator.py', '--drop-first'], timeout=2)
c.start()
try:
```

```
    try:
        c.call_tool('bump', {})
    except Exception as e:
        print('first_error:', e)
    print('after_first:', c.call_tool('get_state', {}))
    print('retry:', c.call_tool('bump', {}))
    print('after_retry:', c.call_tool('get_state', {}))
finally:
    c.close()
PY
```

期望：

```
after_first: count=1
after_retry: count=2
```

8.3 CBAP 幂等对照

```
python3 -m unittest -v tests.cbap.test_conformance.ConformanceTest.test_12_idempotent_write
```

期望：

```
side effect count = 1
```

---

9. 结论

用 CBAP 的四个原语检查后，当前最明确的协议级问题是：

```
MCP 没有定义 lost-response 后的效果去重语义
```

它已经被公开 issue #3394 记录，不应重复提报。

进一步检查字符串/能力目录后，还有三条**安全相关缺口**：

```
P-06 tool 定义没有强制 pin/重批准（已有 SEP #3452）
P-07 server instructions 没有 untrusted 标记/审批
P-08 跨 server tool 名只要求 SHOULD 消歧
```

它们不是 RCE/认证绕过，也不应该被包装成 CVE；

但对“代理所有能发字符串的设备”来说，都是实际攻击面。

CBAP 能做的不是“替代 MCP”，而是把：

```
idempotency
lease
facts
stop
```

变成可执行的验收条件。

如果主人手上的漏洞是另一条，尤其是：

```
SDK 自动重试
OAuth token 透传
跨 server 身份混淆
资源 URI 越界
字符串注入/工具影子
```

请给出具体实现和复现；先判断它是否已经被 #3394 / #3452 或安全指南覆盖，再决定走私密 Security Advisory 还是补 SEP。

---

10. 版本与产物

```
MCP spec commit: dae694f61a31f7a3ec2af3aff709e11f70f61e48
MCP spec date:   2026-10-23
Current spec:    2026-07-28
Issue:           modelcontextprotocol/modelcontextprotocol#3394
CBAP baseline:   commit 1586dfb, audit run 2026-10-24
```

新增：

```
tests/cbap/fake_mcp_mutator.py
docs/MCP-protocol-security-2026-10-24.md
```

未修改 MCP 仓库，未公开提交新的安全 issue。
